16-03-2026 lab 3

ROUND ROBIN ALGORITHM CPU SCHEDULING

CODE:

```
#include <stdio.h>
#include <string.h>
#define MAX 10
#define QUEUE_SIZE (MAX * 10)
typedef struct {
   int pid;
   int at;
   int bt;
   int rt;
   int ct;
   int tat;
   int wt;
} Process;

typedef struct {
   int data[QUEUE_SIZE];
   int front;
   int rear;
} Queue;

void initQueue(Queue *q) {
   q->front = 0;
   q->rear = 0;
}

int isEmpty(Queue *q) {
   return q->front == q->rear;
}

void enqueue(Queue *q, int val) {
   q->data[q->rear++] = val;
}

int dequeue(Queue *q) {
   return q->data[q->front++];
}

void addArrivals(Process procs[], int n, Queue *q, int inQueue[], int currentTime) {
   for (int i = 0; i < n; i++) {
      if (!inQueue[i] && procs[i].rt > 0 && procs[i].at <= currentTime) {
         enqueue(q, i);
         inQueue[i] = 1;
      }
   }
}
```

```
int main() {
    Process procs[MAX];
    Queue readyQueue;
    int inQueue[MAX];
    int n, tq;
    int completed = 0, time = 0;

    printf("=== Round Robin CPU Scheduling ===\n\n");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        procs[i].pid = i + 1;
        printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
        scanf("%d %d", &procs[i].at, &procs[i].bt);
        procs[i].rt = procs[i].bt;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    initQueue(&readyQueue);
    memset(inQueue, 0, sizeof(inQueue));


    addArrivals(procs, n, &readyQueue, inQueue, time);

    printf("\nGantt Chart:\n|");

    while (completed < n) {
        if (isEmpty(&readyQueue)) {

            time++;
            addArrivals(procs, n, &readyQueue, inQueue, time);
            continue;
        }

        int idx = dequeue(&readyQueue);
        Process *p = &procs[idx];

        if (p->rt > tq) {

            printf(" P%d |", p->pid);
            time    += tq;
            p->rt   -= tq;
            addArrivals(procs, n, &readyQueue, inQueue, time);
```

```
            enqueue(&readyQueue, idx);
        } else {

            printf(" P%d |", p->pid);
            time   += p->rt;
            p->rt   = 0;
            p->ct   = time;
            completed++;
            addArrivals(procs, n, &readyQueue, inQueue, time);
        }
    }

    printf("\n\n");


    float totalTAT = 0, totalWT = 0;

    printf("%-8s %-8s %-8s %-8s %-8s %-8s\n",
          "Process", "AT", "BT", "CT", "TAT", "WT");
    printf("-------------------------------------------------\n");

    for (int i = 0; i < n; i++) {
        procs[i].tat = procs[i].ct - procs[i].at;
        procs[i].wt  = procs[i].tat - procs[i].bt;
        totalTAT    += procs[i].tat;
        totalWT     += procs[i].wt;

        printf("P%-7d %-8d %-8d %-8d %-8d %-8d\n",
              procs[i].pid, procs[i].at, procs[i].bt,
              procs[i].ct,  procs[i].tat, procs[i].wt);
    }

    printf("-------------------------------------------------\n");
    printf("Average Turnaround Time : %.2f\n", totalTAT / n);
    printf("Average Waiting Time    : %.2f\n", totalWT  / n);

    return 0;
}
```

Output:

```
=== Round Robin CPU Scheduling ===

Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 2
Enter Arrival Time and Burst Time for P4: 4 1
Enter Time Quantum: 2

Gantt Chart:
| P1 | P2 | P3 | P1 | P4 | P2 | P1 |

Process  AT       BT       CT       TAT      WT
-----------------------------------------------
P1       0        5        12       12       7
P2       1        4        11       10       6
P3       2        2        6        4        2
P4       4        1        9        5        4
-----------------------------------------------
Average Turnaround Time : 7.75
Average Waiting Time    : 4.75
```

2. Write a C program to simulate multi-level queue schedulingalgorithm considering the following scenario. All the
processes in the system are divided into two categories –system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

```
#include <stdio.h>
#define MAX 10
typedef struct {
   int pid, at, bt, type, start, ct, tat, wt, rt;
} Process;
void sortByAT(Process proc[], int idx[], int count) {
   for (int i = 0; i < count - 1; i++)
      for (int j = 0; j < count - i - 1; j++)
         if (proc[idx[j]].at > proc[idx[j + 1]].at) {
            int tmp = idx[j];
            idx[j] = idx[j + 1];
            idx[j + 1] = tmp;
         }
}
int main() {
   int n;
   Process proc[MAX];
   int sysQ[MAX], sysCount = 0;
   int usrQ[MAX], usrCount = 0;
   printf("=== Multilevel Queue Scheduling (System > User, FCFS) ===\n\n");
   printf("Enter number of processes: ");
```

```
scanf("%d", &n);
for (int i = 0; i < n; i++) {
   printf("\nProcess %d (PID AT BT): ", i + 1);
   scanf("%d %d %d", &proc[i].pid, &proc[i].at, &proc[i].bt);
   printf("  Type (0=System, 1=User): "); scanf("%d", &proc[i].type);
   if (proc[i].type == 0) sysQ[sysCount++] = i;
   else                   usrQ[usrCount++] = i;
}
sortByAT(proc, sysQ, sysCount);
sortByAT(proc, usrQ, usrCount);
int time = 0;
printf("\nGantt Chart:\n|");
for (int i = 0; i < sysCount; i++) {
   int p = sysQ[i];
   if (time < proc[p].at) time = proc[p].at;
   proc[p].start = time;
   proc[p].rt    = proc[p].start - proc[p].at;
   printf(" P%d(S) |", proc[p].pid);
   time        += proc[p].bt;
   proc[p].ct    = time;
   proc[p].tat   = proc[p].ct  - proc[p].at;
   proc[p].wt    = proc[p].tat - proc[p].bt;
}
for (int i = 0; i < usrCount; i++) {
   int p = usrQ[i];
   if (time < proc[p].at) time = proc[p].at;
   proc[p].start = time;
   proc[p].rt    = proc[p].start - proc[p].at;
   printf(" P%d(U) |", proc[p].pid);
   time        += proc[p].bt;
   proc[p].ct    = time;
   proc[p].tat   = proc[p].ct  - proc[p].at;
   proc[p].wt    = proc[p].tat - proc[p].bt;
}
printf("\n\n");
float totalTAT = 0, totalWT = 0, totalRT = 0;
printf("%-6s %-6s %-6s %-8s %-6s %-6s %-6s %-6s\n",
      "PID", "AT", "BT", "Type", "CT", "TAT", "WT", "RT");
printf("--------------------------------------------------\n");
for (int i = 0; i < sysCount; i++) {
   int p = sysQ[i];
   printf("%-6d %-6d %-6d %-8s %-6d %-6d %-6d %-6d\n",
         proc[p].pid, proc[p].at, proc[p].bt,
         "System", proc[p].ct, proc[p].tat, proc[p].wt, proc[p].rt);
   totalTAT += proc[p].tat;
   totalWT  += proc[p].wt;
   totalRT  += proc[p].rt;
}
```

```
    for (int i = 0; i < usrCount; i++) {
        int p = usrQ[i];
        printf("%-6d %-6d %-6d %-8s %-6d %-6d %-6d %-6d\n",
               proc[p].pid, proc[p].at, proc[p].bt,
               "User", proc[p].ct, proc[p].tat, proc[p].wt, proc[p].rt);
        totalTAT += proc[p].tat;
        totalWT  += proc[p].wt;
        totalRT  += proc[p].rt;
    }
    printf("--------------------------------------------------\n");
    printf("Average Turnaround Time : %.2f\n", totalTAT / n);
    printf("Average Waiting Time    : %.2f\n", totalWT  / n);
    printf("Average Response Time   : %.2f\n", totalRT  / n);
    return 0;
}
```

Output:

```
=== Multilevel Queue Scheduling (System > User, FCFS) ===

Enter number of processes: 4

Process 1 (PID AT BT): 1 0 4
  Type (0=System, 1=User): 0

Process 2 (PID AT BT): 2 2 3
  Type (0=System, 1=User): 0

Process 3 (PID AT BT): 3 1 5
  Type (0=System, 1=User): 1

Process 4 (PID AT BT): 4 2 3
  Type (0=System, 1=User): 1

Gantt Chart:
| P1(S) | P2(S) | P3(U) | P4(U) |

PID    AT     BT     Type     CT     TAT    WT     RT
--------------------------------------------------
1      0      4      System   4      4      0      0
2      2      3      System   7      5      2      2
3      1      5      User     12     11     6      6
4      2      3      User     15     13     10     10
--------------------------------------------------
Average Turnaround Time : 8.25
Average Waiting Time    : 4.50
Average Response Time   : 4.50
```